# ONBRD-00: Architect's Sequence & Dependency Runbook

> **Series:** ONBRD — Dynatrace Onboarding | **Reference:** 00 — Architect's Sequence & Dependency Runbook | **Created:** May 2026 | **Last Updated:** 10/19/2026

## Overview

Sequence-and-dependency runbook for an architect or experienced tenant lead onboarding a new Dynatrace tenant in 2026. Assumes you have onboarded a tenant before — this is order, dependencies, decisions, not a GUI tour.

Per-domain depth lives in the canonical series (linked at each step). For recommended defaults, the decision matrix, validation DQL, anti-patterns, and the cross-series next-steps map, see **ONBRD-99: Best Practice Summary**.

---

## Table of Contents


1. The Sequence
2. Phase 1 — Foundation
3. Phase 2 — Organize
4. Phase 3 — Operate
5. Phase Checkpoints
6. Common Variants & Skip Rules
7. After ONBRD

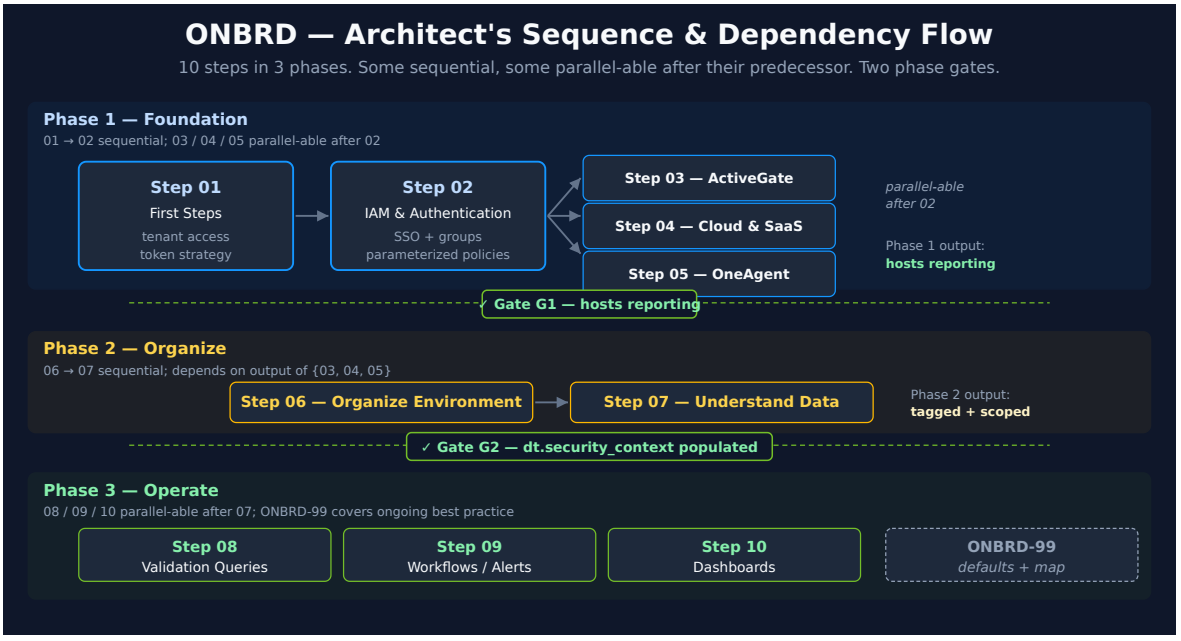

---

# Prerequisites

| Requirement | Details |
|---|---|
| Tenant URL + admin credentials | from procurement / account team |
| Account-level decisions in place | tenant region, SaaS vs Managed, account hierarchy. If not yet decided, see **NR2DT-00: Tenant Prerequisites — Minimum Setup Before Migration** — the same checklist generalizes beyond New Relic migrations. |
| Reading time | 30–45 minutes for the runbook; the actual onboarding is 1–4 weeks depending on scope and skip rules applied. |

# 1. The Sequence

ONBRD is 10 steps grouped into 3 phases. Some are sequential, some parallel-able after their predecessor. Two phase gates separate the phases.




| Phase | Steps | Phase Output |
|---|---|---|
| **Phase 1 — Foundation** | 01 → 02 → {03, 04, 05} | Hosts reporting; tenant lead can query data |

| Phase | Steps | Phase Output |
|---|---|---|
| **Phase 2 — Organize** | 06 → 07 | Entities tagged and scoped; `dt.security_context` populated |
| **Phase 3 — Operate** | {08, 09, 10} | Validation DQL set + at least one workflow + at least one dashboard |

# 2. Phase 1 — Foundation

## Step 01 — First Steps

- **Decision:** Confirm access; record tenant URL; plan token strategy for downstream automation.
- **Depends on:** procurement complete, tenant provisioned.
- **2026 specifics:** Plan for Platform Tokens ( `dt0s16` , `Authorization: Bearer` ) as the default for new automation — the access-token upgrade guide states that *"In Latest Dynatrace, this model is replaced with platform tokens"*. Settings v2 / Configuration as Code (Terraform / Monaco) over Configuration API for new work (SaaS 1.337 put deprecation notices on Configuration API endpoints now covered by Settings). **Extensions 2.0** for any custom integrations (Extensions Framework 1.0 reached end of support on 2025-03-31).
- **Output:** confirmed tenant access; first Platform Token decision recorded.
- **Deep dive: ONBRD-99: Best Practice Summary** § 1 Recommended Defaults for the 2026 token / API / extension stack.

## Step 02 — IAM and Authentication

- **Decision:** SSO method (SAML, optionally with SCIM provisioning); initial group structure; first parameterized policies.
- **Depends on:** Step 01 (access confirmed).
- **2026 specifics:** Issue the first Platform Token ( `dt0s16` , `Authorization: Bearer` ). Installer downloads work with a platform token carrying `fleet-management:oneagents:download` (OneAgent) or `fleet-management:activegates:download` (ActiveGate); classic access tokens ( `dt0c01` , `InstallerDownload` scope) exist only on classic and hybrid environments — *"Classic access tokens don't exist in latest environments"*. Use parameterized policies bound to

groups via binding parameters; the `dt.security_context` field is the standardized boundary for Gen3 IAM scoping (configured fully in Step 06).

- **Output:** SSO live; admin group + first parameterized policy in place; first Platform Token issued.
- **Deep dive:** IAM series for full IAM administration (15 notebooks); **FAQ-02: Tagging — Sources, Standards, and Strategy** for `dt.security_context` strategy.

## Step 03 — ActiveGate (parallel-able with 04 + 05 after 02)

- **Decision:** Need ActiveGate? Needed when hosts have no direct egress, for private synthetic locations, remote extensions, Kubernetes monitoring and classic AWS / Azure polling; otherwise routing OneAgent traffic through an ActiveGate is a trade-off against going direct (see the decision table and the routing trade-offs in **ONBRD-03: Deploying ActiveGate** § 2). Defer if every host can reach your environment and nothing else needs one.
- **Depends on:** Step 02 (Platform Token to register AG).
- **2026 specifics:** Disk 10–20 GB (sizing depth in **ONBRD-03** and **FAQ-10: How Do I Size and Scale ActiveGates?**); 2–3 AGs per zone for load distribution + failover.
- **Output:** at least one AG registered; cluster connectivity verified; AG visible in **Fleet Management** (or via `smartscapeNodes "ACTIVEGATE"`).
- **Deep dive:** CLOUD series for AG-routed cloud integrations; AUTOM series for AG config-as-code.

## Step 04 — Cloud & SaaS Integrations (parallel-able with 03 + 05 after 02)

- **Decision:** Which clouds (AWS / Azure / GCP) and integration mechanism per cloud.
- **Depends on:** Step 02 (Platform Token); optionally Step 03 (AG for legacy paths).
- **2026 specifics:** Clouds app for AWS (GA) and Azure (direct connection since SaaS 1.337 — no ActiveGate for metric polling); the GCP Clouds-app connection is in **Preview** — verify it has reached your tenant; until then the classic GCP integration (`dynatrace-gcp-monitor` on GKE or as a Cloud Function) remains the working path. It sends straight to your environment URL and needs no ActiveGate on SaaS. AWS Lambda primary-field propagation to logs and spans via the `DT_TAGS` env var (OneAgent 1.337). IAM Role pattern for production multi-account.
- **Output:** cloud entities visible in Smartscape, with cloud tags on the nodes (for example the `tags:aws` record on AWS nodes); on signals, cloud tags appear as

`aws.tags.<key>` / `azure.tags.<key>` / `gcp.labels.<key>` only for keys you add to the tag-enrichment configuration.

- **Deep dive:** CLOUD series for per-provider depth; **FAQ-02: Tagging — Sources, Standards, and Strategy** for cross-cloud tag normalization.

## Step 05 — OneAgent Deployment (parallel-able with 03 + 04 after 02)

- **Decision:** Deployment method (direct, package manager, Operator for K8s); initial scope (pilot or full); host-tag taxonomy.
- **Depends on:** Step 02 (Platform Token with `fleet-management:oneagents:download` for installer download).
- **2026 specifics: Set primary fields/tags at install** via the single `--set-host-tag` form the June-2026 tags hub documents for both: `oneagentctl --set-host-tag="primary_tags.<key>=<value>"` (prefix written explicitly — it is never added automatically) and `oneagentctl --set-host-tag="dt.security_context=<value>"` — or later on existing hosts, since the enrichment page documents an option to *"Define fields and tags for existing OneAgent installations using oneagentctl"* (in community practice, expect the new fields only on data ingested after the change). Primary tags emit on every signal (metrics / spans / logs / events) at ingest. Kubernetes: Dynatrace Operator + **Cloud Native FullStack** mode (the Operator docs list Classic Full-Stack under *Other* deployment modes; in community practice, prefer Cloud Native FullStack for new clusters). DynaKube **v1beta6** for new manifests — the Operator 1.11 release notes say *"Before upgrading, update all DynaKube manifests to v1beta6."* From **OneAgent 1.337**, Windows OneAgent requires **Npcap** for Network Agent metrics (WinPcap is no longer supported).
- **Output:** OneAgent installed on at least one host group; primary tags propagating to every signal at ingest.
- **Deep dive:** K8S series for DynaKube + Cloud Native FullStack; **FAQ-01: Why you need a good Host Group naming strategy** for host-group naming; **FAQ-02: Tagging — Sources, Standards, and Strategy** for primary-fields strategy; **FAQ-03: OneAgent and OpenTelemetry — Choosing Per Workload** for OneAgent and OTel decision.

> **Sources:**
>
> - Upgrade from access tokens classic (DT docs) — *"Latest environment Platform tokens only. Classic access tokens don't exist in latest environments, and v2/apiTokens isn't available."*
> - Download latest OneAgent installer (DT docs) — *"Platform Token / OAuth: Required scope: fleet-*

> *management:oneagents:download"*
>
> - Download latest ActiveGate installer (DT docs) — *"Platform Token / OAuth: Required scope: fleet-management:activegates:download"*
> - SaaS 1.337 release notes (DT docs) — *"Certain Configuration API endpoints are now deprecated"*
> - OneAgent 1.337 release notes (DT docs) — *"Primary fields set via DT_TAGS are now added to logs and spans in AWS Lambda deployments of OneAgent."*
> - Enterprise identity management (DT docs), Fleet Management (DT docs), Primary Grail fields and tags enrichment (DT docs), Kubernetes deployment modes (DT docs), Operator 1.11.0 release notes (DT docs)
> - **Dictionary:** `aws.tags.__tag_key__` ( `experimental` ), `azure.tags.__tag_key__` ( `experimental` ), `gcp.labels.__label__` ( `experimental` ), read 10/02/2026; no singular `aws.tag.*` / `azure.tag.*` / `gcp.label.*` fields under the same `startsWith` filter (control: `tags` , `stable` , returned).

# 3. Phase 2 — Organize

## Step 06 — Organizing Your Environment

- **Decision:** Tag taxonomy (env / team / app / compliance / cost-center dimensions); host-group naming convention; bucket strategy; segment definitions.
- **Depends on:** Step 05 (OneAgent hosts reporting); Step 04 (cloud tags arriving) if cross-cloud normalization is needed.
- **2026 specifics:** Primary fields/tags via `oneagentctl --set-host-tag="primary_tags.<key>=<value>"` and `--set-host-tag="dt.security_context=<value>"` (the same `--set-host-tag` form the oneagentctl reference uses for `dt.security_context` ). Use **Segments + `dt.security_context`** for data scoping — *not* legacy Management Zones. API 1.337 (April 2026) marks the `managementZone` and `serviceTag` properties of calculated service metric definitions as deprecated. Bucket strategy: `dt.security_context` is the default for general data access; buckets only for compliance / retention / hard cost / hostile multi-tenancy.
- **Output:** entities grouped meaningfully; `dt.security_context` populated on signals; segments defined for primary scopes; bucket strategy documented.
- **Deep dive:** ORGNZ series — buckets, segments, `dt.security_context` (full depth, 11 notebooks); IAM series — parameterized policies bound to `dt.security_context` ;

**FAQ-01: Why you need a good Host Group naming strategy** — host-group naming; **FAQ-02: Tagging — Sources, Standards, and Strategy** — tagging strategy.

### Step 07 — Understanding Your Data

- **Decision:** Which signals matter for the workloads in scope (spans / logs / metrics / events / business events). Confirm primary fields propagate.
- **Depends on:** Step 06 (entities grouped); Step 05 (OneAgent producing signals).
- **2026 specifics:** OneAgent attribute enrichment (OneAgent 1.333+) emits primary fields on every signal at ingest — spot-check `dt.security_context` on logs / spans / metrics with the validation DQL in **ONBRD-99: Best Practice Summary** § 3.
- **Output:** confirmed which signal types are flowing; one validation DQL per signal type saved.
- **Deep dive:** SPANS series, OPLOGS series, OPIPE series, OPMIG series — per-signal-type depth.

> **Sources:** OneAgent command-line configuration (DT docs) — *"./oneagentctl --set-host-tag=dt.security_context=easytrade_sec"*; Dynatrace API 1.337 changelog (DT docs) — *"Changed property managementZone Deprecated changed to true"*.

## 4. Phase 3 — Operate

### Step 08 — Validation Queries (parallel-able with 09 + 10 after 07)

- **Decision:** Which canonical queries become the deployment's validation set (host count, service inventory, log volume per source, primary-field propagation).
- **Depends on:** Step 07 (signals understood); Step 06 (entities grouped).
- **Output:** validation DQL set saved as a notebook or dashboard (the architect's smoke-test for the tenant).
- **Deep dive: ONBRD-99: Best Practice Summary** § 3 Validation Queries for the canonical set; SPANS, DBMON, WEBRUM, BIZEV for per-domain DQL patterns.

### Step 09 — Workflows & Alerts (parallel-able with 08 + 10 after 07)

- **Decision:** Which Davis-detected problems become workflows; which custom anomaly detectors are needed; notification targets; on-call routing.
- **Depends on:** Step 07 (data flowing); Step 06 (entities grouped for scoping).
- **2026 specifics:** Workflows over Alerting Profiles. Davis Anomaly Detectors over legacy metric-events (where the signal is supported).
- **Output:** at least one workflow with notification target; first alert fires.
- **Deep dive:** WFLOW series — workflows + notifications; AIOPS series — Davis problems + anomaly detection + Davis CoPilot.

### Step 10 — Dashboards (parallel-able with 08 + 09 after 07)

- **Decision:** Tier strategy (Executive / Operations / Engineering); per-tier dashboard structure; sharing model.
- **Depends on:** Step 07 (data understood); Step 06 (entities grouped for filters).
- **Output:** at least one dashboard per tier in scope; shared with the right groups.
- **Deep dive:** DASH series — dashboard strategy + executive reporting + tile-type decision.

## 5. Phase Checkpoints

Verify before moving from one phase to the next. The DQL queries in **ONBRD-99: Best Practice Summary** § 3 Validation Queries make these gates concrete.

### G1 — Foundation → Organize

The tenant has at least one host reporting and the tenant lead role can query that data.

- *Verify:* `smartscapeNodes "HOST"` returns host count > 0; tenant lead role can run that query.
- *If not:* stay in Phase 1 — most likely the OneAgent installer didn't run, or the installer-download token lacks its scope: `fleet-management:oneagents:download` on a platform token, or `InstallerDownload` on a classic access token (classic and hybrid environments only).

### G2 — Organize → Operate

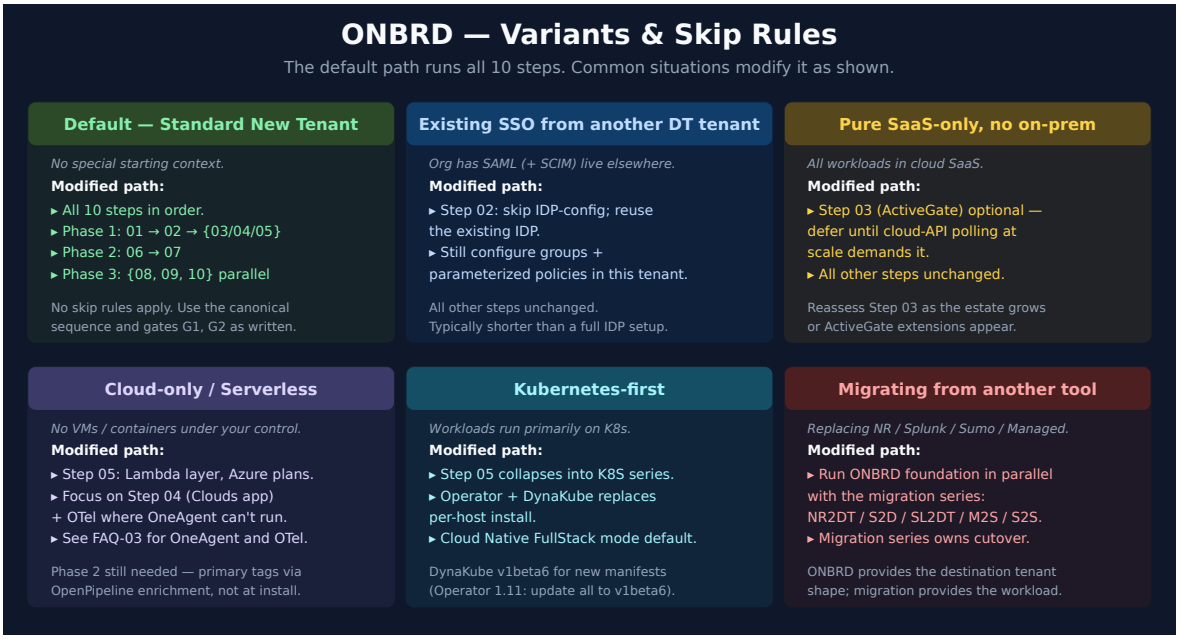
Entities are grouped meaningfully and `dt.security_context` is populated on signals.

- *Verify:* `fetch logs, from:-1h | summarize count(), by:{dt.security_context}` returns at least one non-null value.
- *If not:* OneAgent install or OpenPipeline enrichment didn't propagate the field — fix before Phase 3 (filter scoping in dashboards / alerts will not work without it).

### After Phase 3 — exit ONBRD

ONBRD ends. The next checkpoint is *"do you need a deeper series for the domains in scope?"* — see **ONBRD-99: Best Practice Summary** § 5 Where to Go Deeper for the cross-series reference map.

## 6. Common Variants & Skip Rules




| Variant | When | What to do |
|---|---|---|
| Existing SSO already configured | New tenant for an org with SAML (optionally + SCIM) live on another Dynatrace tenant | Skip the IDP-config part of Step 02; reuse the existing IDP. Still configure groups + parameterized policies in the new tenant. |

| Variant | When | What to do |
|---|---|---|
| Pure SaaS-only, no on-prem | All workloads in cloud; no private/on-prem footprint | Step 03 (ActiveGate) is optional — defer until cloud-API polling demands it. |
| Cloud-only, no hosts | Serverless / managed services only; no VMs or containers under your control | Step 05 narrows rather than disappears: OneAgent reaches AWS Lambda through the Dynatrace Lambda layer (Python, Node.js, Java, .NET) and supported Azure Functions plans from OneAgent 1.343. Use OpenTelemetry for application code where OneAgent does not run (for example GCP Cloud Functions) or where OTel is already your strategy. Focus on Step 04 (cloud integrations) either way. See **FAQ-03: OneAgent and OpenTelemetry — Choosing Per Workload**. |
| Kubernetes-first | Workloads run primarily on K8s | Step 05 collapses into K8S series — Operator + DynaKube replaces per-host install. |
| Migrating from another tool | Replacing New Relic / Splunk / Sumo Logic / managed Dynatrace | Run ONBRD foundation in parallel with the migration series (NR2DT, S2D, SL2DT, M2S, S2S). |
| Multi-tenant org | Several Dynatrace tenants for different BUs / environments | Step 02 expands — see **IAM-08: Multi-Environment IAM** for cross-tenant patterns. |

> **Sources:** AWS Lambda integration (DT docs) — *"using auto-instrumentation without code changes"*; OneAgent 1.343 release notes (DT docs) — Azure Functions: Python on the Flex Consumption Plan for Linux; Java and Node.js on Consumption, Premium, and Dedicated plans for Windows.

## 7. After ONBRD

ONBRD ends after Phase 3. The next set of decisions is per-domain — observability, automation, AI/Davis, dashboards-at-scale, etc.

Reference points to keep open:

- **ONBRD-99: Best Practice Summary** — the architect's reference card: recommended defaults, decision matrix, validation queries, anti-patterns, and the **cross-series reference map** for what to do after ONBRD.
- **-START-HERE- › Foundation Module** — cross-series Foundation Module reading order (ONBRD + ORGNZ + IAM in parallel) with priorities and skip rules.
- **-START-HERE- › Doorway 1 — Net New** — doorway selection for greenfield vs migration paths (Sub-Paths A–E).
- **ADOPT series** — overall maturity and adoption roadmap once ONBRD is complete.

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against the current Dynatrace documentation.*